Praktikum 6: Global Illumination

The last Praktikum task is about adding global lighting effects. The hard-working programmer even added multithreading over the festive period so that it now runs a little faster. So far, the local colour, reflection and refraction have been determined deterministically. For realistic images, however, global lighting effects must be taken into account. Sampling is used for this. When a ray hits an object, the colour at this point is determined by shooting a number of sampled rays into the scene. The more samples, the more realistic the image point.

Good luck!

If you have any questions or problems, please write to us in Moodle!

Aufgabe 1 — Global Illumination

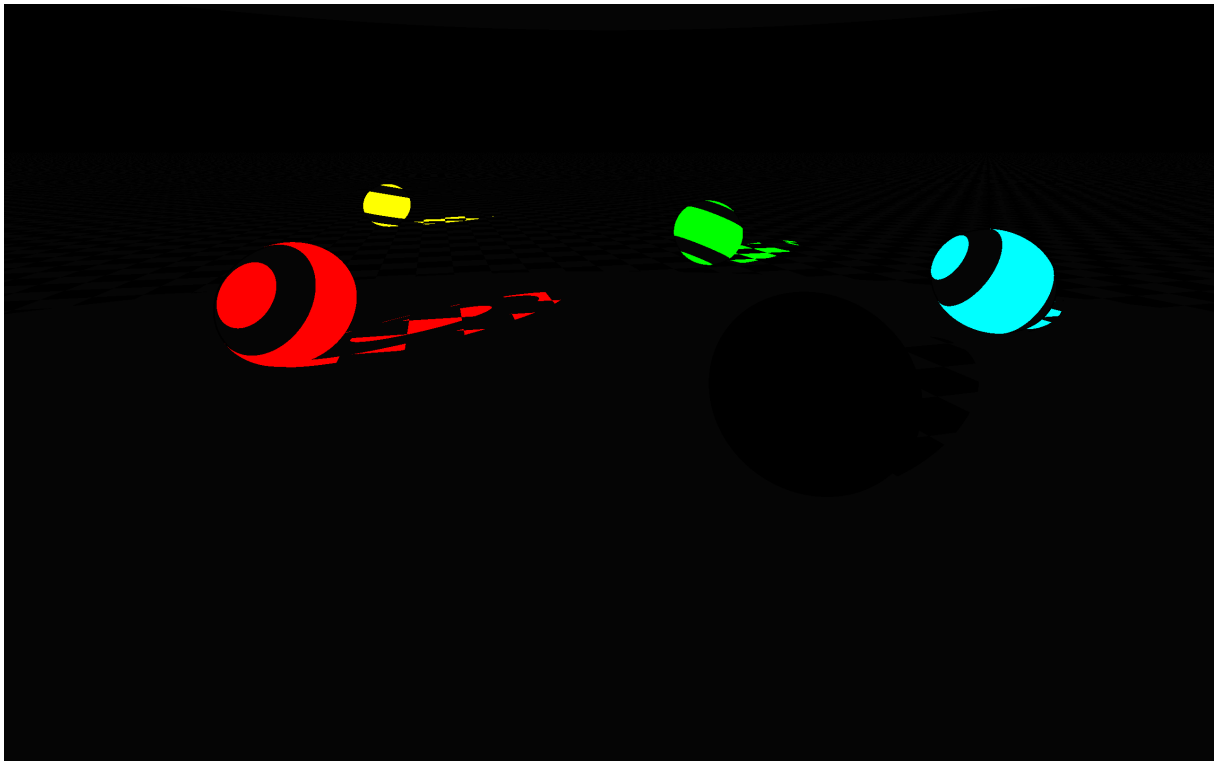


Figure 1: This is what you should see if you run the code without changing anything.

a) Open https://git.uni-due.de/vs.ude/objektorientierte-programmierung-cpp. Use git to *clone* the repository and import the project into 06_GlobalIllumination/.

b) In `Scene.cpp` you will find the method `vec3 get_random_direction`, which determines a random vector for a normal vector (vector which always points away from a surface), which points into the hemisphere around the normal vector. Currently it is a bad random vector generator as it always returns the same value. Change the code so that a vector with 3 random values, each between –1.0 and 1.0, is returned.

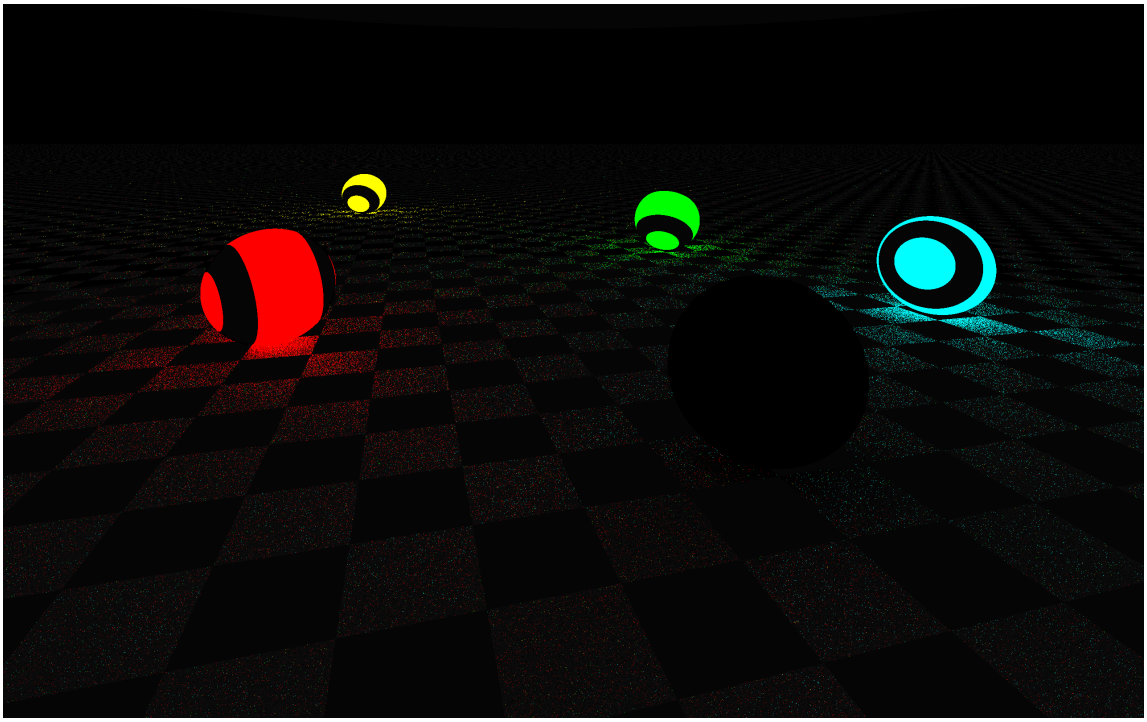


Figure 2: It should look like this (or something similar) if the sampling has been implemented correctly.

c) In `Scene.cpp` you will find the method `vec3 Scene::traceRayRealistic`, which is intended to implement realistic ray tracing. It is called in parallel in `YourRayTracer::renderRealistic` or `YourRayTracer::render_row`. Take a look at `Scene::traceRay` to consider how `reflection` should be calculated so that the reflection in the spheres and the surface looks realistic.

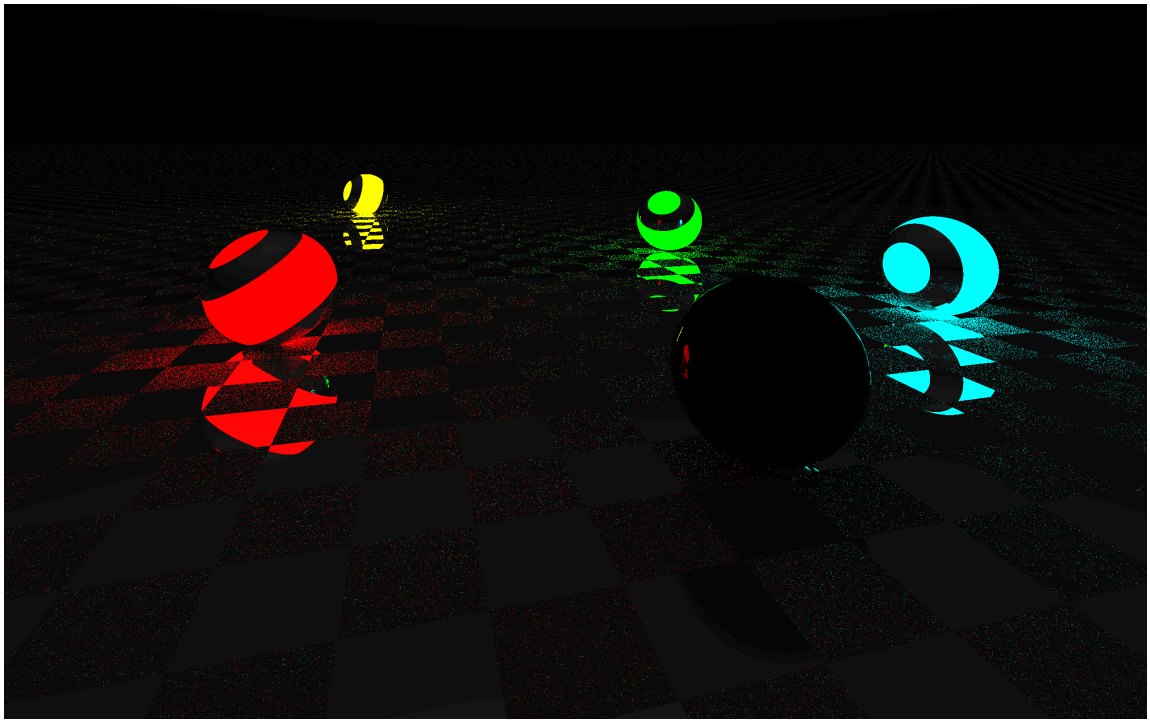


Figure 3: This (or something similar) is how it should look when reflections have been implemented correctly.

d) The same for the refraction. You can see what it should look like in the glass sphere at the front.

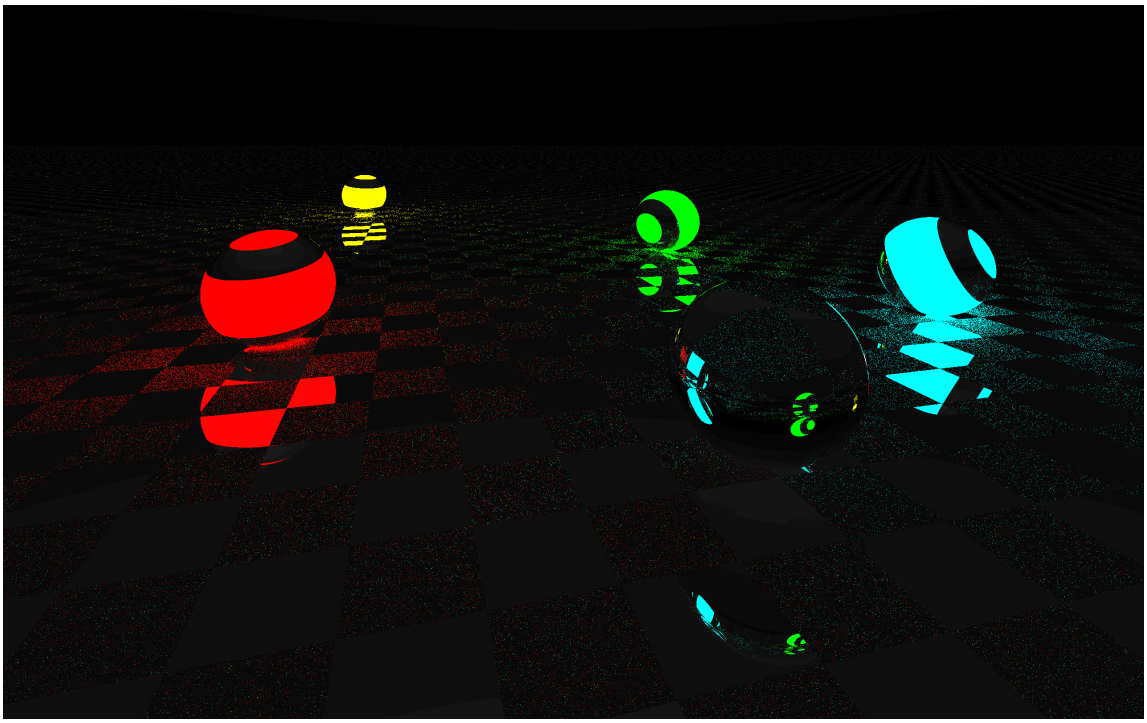


Figure 4: This (or something similar) is what it should look like if refractions have been implemented correctly.

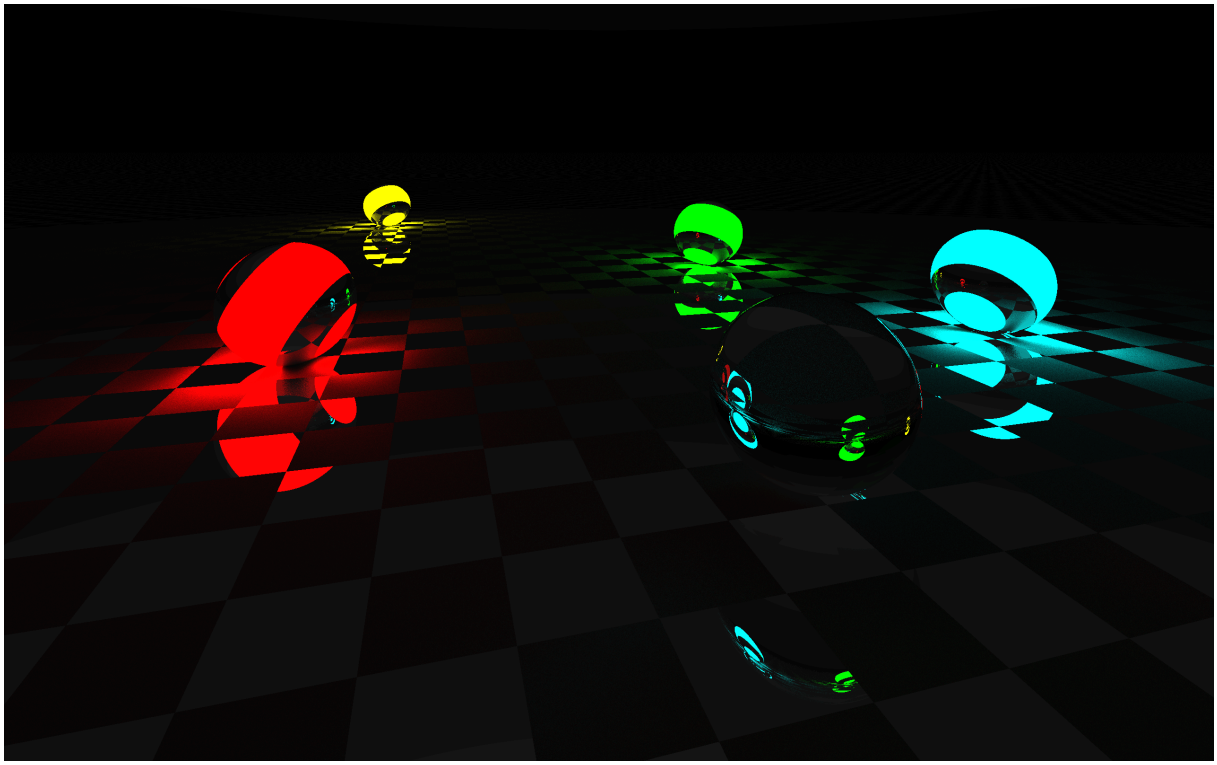


Figure 5: In `main.cpp` you can set how many samples are to be used when creating the `Scene`. With 1000 samples, it may look like this.

If you have made it this far, congratulations! I hope you have enjoyed this internship and that you have learnt something for your further education. Computer graphics is a huge field and there are countless optimisations that could be made to make the images look even more beautiful or realistic. I wish you every success in preparing for the exam.